MPI-LLM: Extreme-scale Multi-LLM Serving for HPC

Scalable LLM Serving and Switching with Lightweight MPI Control

Shu Shi, Wenyi Wang, Yadu Nand Babuji, Kyle Chard, Ian Foster
Department of Computer Science, University of Chicago · Globus Labs

Abstract

Large-scale concurrent LLM serving on HPC systems requires deploying, serving, and switching many model instances across accelerator-rich allocations while handling concurrent request streams. While existing serving systems provide high-performance inference engines, HPC platforms impose different constraints and opportunities: batch allocations, restricted services, explicit placement, and highly optimized MPI communication.

We introduce MPI-LLM, a lightweight MPI-native serving system designed for the HPC ecosystem. MPI-LLM launches persistent scheduler and compute workers through the HPC job system and coordinates concurrent request dispatch, model deployment, shutdown, and switching through MPI. By building directly on the HPC execution model, MPI-LLM supports fast model lifecycle management, dynamic model switching, and large-scale concurrent serving. Our evaluation on Aurora shows scalable model deployment and weak-scaling throughput compared with a LiteLLM[2] + Ray Serve[3] baseline.

Key Designs

- **MPI-native control plane**
  Use MPI as the lightweight serving control substrate for HPC allocations.
- **Persistent accelerator workers**
  Keep scheduler and compute workers alive as MPI ranks across model assignments.
- **Explicit lifecycle commands**
  Represent setup, shutdown, switching, and dispatch as scheduler-controlled MPI actions.
- **Dynamic model switching**
  Support fast online switching through the MPI control plane.
- **Global queue + worker/model state**
  Maintain pending requests and model residency state in the MPI Master for centralized scheduling.

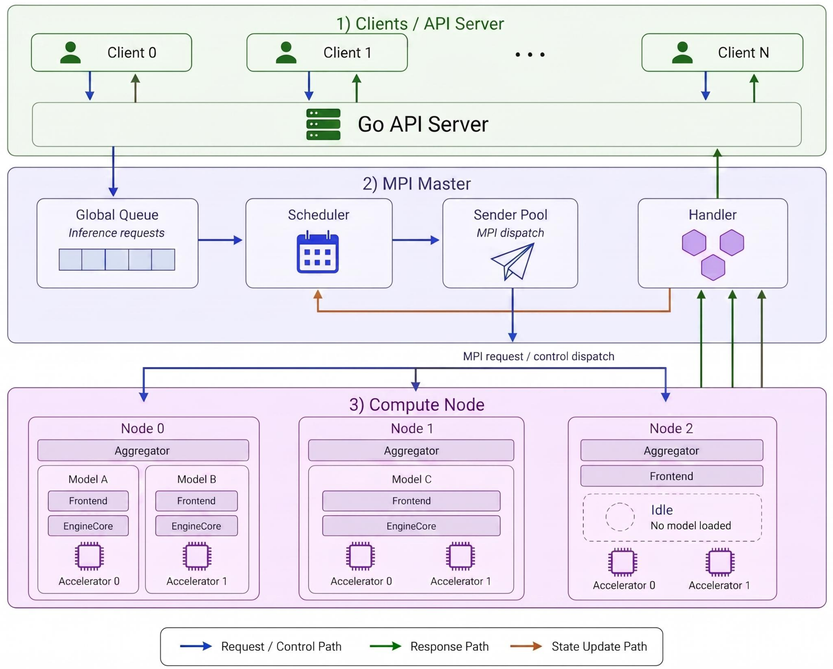
Takeaways

- MPI-LLM provides a lightweight MPI-native control plane for large-scale LLM serving on HPC systems.
- On Aurora, MPI-LLM maintains stable model deployment time and sustains weak-scaling throughput at large scale.
- MPI-LLM also supports fast model switching, achieving sub-5s switching for Llama-8B on Polaris.

References

[1] W. Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP '23.
[2] LiteLLM. Open-source AI gateway and proxy server. https://docs.litellm.ai.
[3] P. Moritz et al. 2018. Ray: A Distributed Framework for Emerging AI Applications. Proc. OSDI '18.

MPI-LLM System Architecture



System Architecture

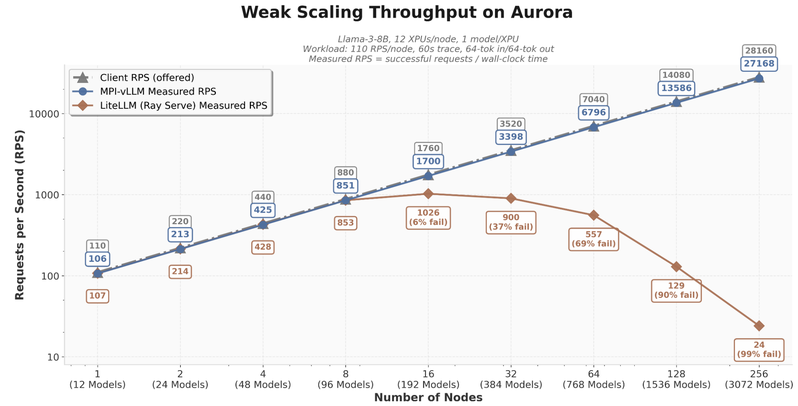
MPI-LLM separates the serving stack into an MPI Master on the service node and persistent compute nodes for concurrent model execution. The MPI Master maintains the global queue, scheduling state, and dispatch path, while compute nodes keep worker-side runtime components alive across model assignments. Responses and scheduler state are aggregated through the Handler path, allowing the system to coordinate many model instances and concurrent requests without repeatedly restarting workers.

Component Roles

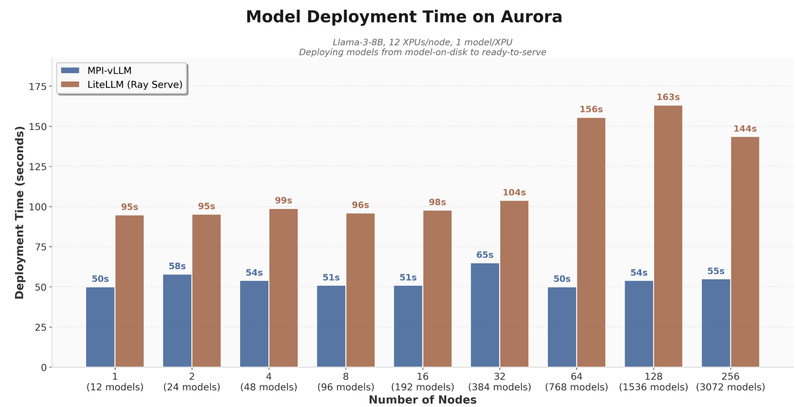
- **Go API Server**
  Receives client requests, tracks in-flight requests, and returns completed responses.
- **MPI Master**
  Maintains the global request queue, worker/model state, and scheduling decisions.
- **Scheduler**
  Assigns requests to model instances and generates setup, shutdown and switching actions using global queue and worker state.
- **Sender Pool**
  Dispatches requests and lifecycle control actions to compute nodes through MPI.
- **Handler**
  Collects results from aggregators and pushes state updates to the Scheduler.
- **Aggregator**
  Receives and routes requests to model Frontends and returns completed outputs.
- **Frontend**
  Manages per-model serving logic and interfaces with vLLM[1].
- **EngineCore**
  Runs vLLM continuous batching and model execution.

Experiments

We evaluate weak scaling on Aurora's Intel XPU nodes by assigning one Llama-3-8B model instance to each XPU and increasing the number of nodes while keeping the offered load fixed at 110 RPS per node. MPI-LLM closely tracks the offered load as scale increases, while the LiteLLM + Ray Serve baseline suffers increasing failure rates and eventually collapses at large scale.



We also measured the time to deploy Llama-3-8B model instances from model-on-disk to ready-to-serve across Aurora nodes. MPI-LLM keeps deployment time stable as the number of model instances grows to 3072, while LiteLLM + Ray Serve deployment time increases substantially at larger scales.



Toward Dynamic Model Switching

With a global request queue and a lightweight MPI control plane, MPI-LLM supports fast model switching during online serving. On Polaris, MPI-LLM achieved sub-5s switching for Llama-8B, highlighting its potential for dynamic multi-model serving.

Acknowledgement: This research used resources of the Argonne Leadership Computing Facility, which is a U.S. Department of Energy Office of Science User Facility operated under contract DE-AC02-06CH11357.